



POLITECNICO
MILANO 1863



ITECNICO
LANO 1863

Architettura dei calcolatori e sistemi operativi

Gerarchia di memoria

Capitolo 5 P&H

Problema - è difficile realizzare memorie che siano molto grandi e molto veloci

Esistono attualmente diverse tecnologie per realizzare le memorie, ma nessuna tecnologia ci permette di soddisfare allo stesso tempo i due *requisiti fondamentali ipotizzati nella costruzione del processore*:

- accedere a una parola di memoria in un solo ciclo di clock del processore
- essere così grande da contenere qualsiasi programma (cioè avere uno spazio di indirizzamento fisico uguale allo spazio di indirizzamento virtuale)

Per superare questo limite si realizza una
“*gerarchia di memoria*”



- Consideriamo 2 tecnologie di memoria:
 - tecnologia A: permette di realizzare memorie più piccole e veloci
 - tecnologia B: permette di realizzare memorie più grandi ma più lente
 - Idea base:
 - tenere il programma nella memoria B e portare nella memoria A solo le porzioni di programma che servono durante una certa fase di esecuzione
 - Questa idea funziona solamente se il programma usa relativamente a lungo la porzione di programma contenuta nella memoria veloce prima di richiedere di sostituirla con un'altra
- è funziona perché i programmi sono generalmente temporalmente e spazialmente locali

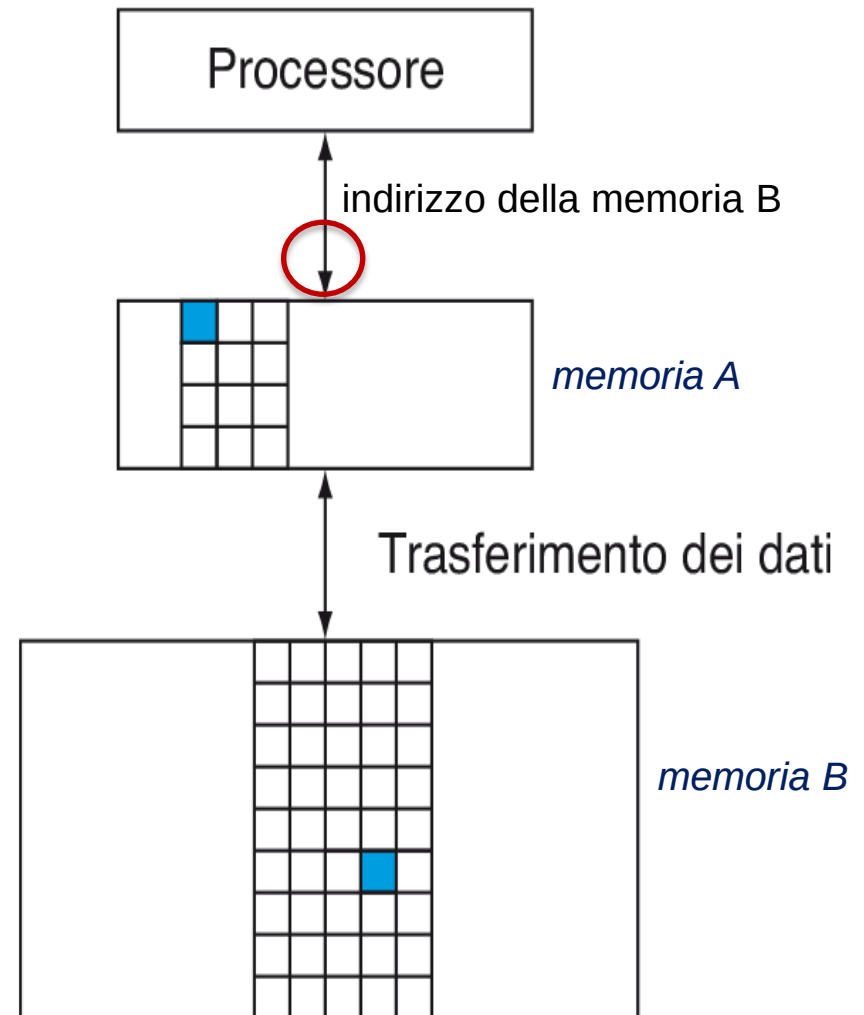
Una gerarchia a 2 livelli

Il processore genera indirizzi della memoria B

Deve essere possibile derivare da un indirizzo della memoria B il “corrispondente” della memoria A

Il processore lavora sulla memoria A, più piccola e veloce

Quando richiede un indirizzo che risulta non presente in memoria A, un intero blocco viene trasferito dalla memoria B alla memoria A



Località: comportamento dell'accesso a memoria dei programmi

La distribuzione degli accessi di un programma alla memoria nello spazio e nel tempo non è omogenea, ma presenta la caratteristica di **località**

- **Località temporale**: indica che un programma tende ad accedere agli stessi indirizzi di memoria cui ha già acceduto di recente (cicli di istruzioni)
- **Località spaziale**: indica che un programma accede con maggior probabilità agli indirizzi vicini a quelli cui ha già acceduto in passato (strutture dati lineari)

Questo comportamento è esibito praticamente da tutti i programmi e rende efficace l'idea base della gerarchia di memoria



Funzionamento dell'idea base -1

L'idea base enunciata si concretizza nel comportamento seguente:

- Carica il programma nella memoria B
- Quando il programma **richiede una parola** a un certo indirizzo, trasferisci il **BLOCCO** di parole che contiene la parola all'indirizzo richiesto dalla memoria B alla memoria A (*blocco* per località spaziale)
- Grazie alla località spaziale e temporale il programma chiederà di accedere molte volte a parole del blocco caricato in memoria A; questi accessi sono detti **HIT**
- Prima o poi il programma chiederà di accedere a un indirizzo non presente in memoria A; questo accesso è detto **MISS**
- Quando si verifica un MISS un nuovo blocco, contenente la parola a cui accedere, viene caricato dalla memoria B alla memoria A



Funzionamento dell'idea base - 2

- La memoria veloce non è costituita da un unico blocco, ma da molti blocchi, perchè la **località temporale può riguardare dati non contigui in memoria**
- Fino a quando la memoria veloce ha blocchi liberi, in risposta a una MISS viene caricato un nuovo blocco nello spazio libero
- Quando però la memoria veloce è tutta occupata, per caricare un blocco è necessario scegliere un blocco già caricato da liberare
- La modalità di scelta del blocco da liberare è detta **politica di sostituzione**



Politica di sostituzione

E' possibile definire diverse politiche di sostituzione

➤ Casuale

Statisticamente migliore ma computazionalmente inefficiente

- **Least Recently Used** (LRU): sostituisce il **blocco usato meno di recente**, perché, in base al principio di **località temporale**, questo ha la probabilità più bassa di essere richiesto nel prossimo futuro

- **FIFO** (First In First Out): sostituisce sempre il blocco **caricato meno di recente**, indipendentemente da quando vi sia stato fatto riferimento (cioè sia stato usato)

Statisticamente il comportamento migliore sarebbe ottenuto con LRU, però la realizzazione esatta di LRU è troppo difficile, quindi si utilizzano approssimazioni di diverso tipo in base al livello di gerarchia



Condizioni di funzionamento accettabile della gerarchia

Parametri relativi all'accesso di un programma alla memoria organizzata su due livelli

- **h = HIT rate** (tasso di hit): è la percentuale di accessi che vengono soddisfatti dalla memoria A ($n^{\circ}HIT/n^{\circ}totale\ accessi$)
- **m = MISS rate** (tasso di miss): è la percentuale di accessi che ha richiesto di caricare un nuovo blocco dalla memoria B alla memoria A ($n^{\circ}MISS/n^{\circ}totale\ accessi$)

chiaramente: $m = 1 - h$

- **M = MISS penalty**, è il tempo richiesto per svolgere l'operazione di *caricamento* di un blocco dal livello di memoria più lento

L'idea base funziona in pratica se la **località dei programmi** e la **dimensione della memoria veloce** permettono di raggiungere HIT rate sufficientemente elevati da compensare ampiamente le MISS penalty



Gerarchie a più livelli

- Attualmente i programmi utilizzano spazi di indirizzamento (virtuali) estremamente grandi e i processori hanno bisogno di memorie così veloci che una gerarchia a 2 livelli non permette di soddisfare i requisiti
- Si costruiscono quindi gerarchie di memoria con più di 2 livelli; per ogni coppia di livelli si applicano *in linea di principio* le considerazioni fatte per la gerarchia a 2 livelli

I meccanismi differiscono nei dettagli, perchè tra coppie di

- ai livelli più in alto nella gerarchia (*...l'altezza massima è quella dei registri dentro la CPU*) si devono usare meccanismi di accesso veloci (*totalmente HW*), mentre
- ai livelli più bassi si possono usare meccanismi più lenti (*che includono interventi del Sistema Operativo*)



Tecnologie di memoria

Attualmente le tecnologie utilizzate per la realizzazione della gerarchia di memoria sono, in ordine di velocità decrescente e dimensione crescente:

- SRAM (Static Random Access Memories) - volatile
- DRAM (Dynamic Random Access Memories) - volatile
- Flash – non volatile
- Dischi Magnetici – non volatile

livello	velocità	tipiche dimensioni
SRAM	0,2 – 2,5 ns	decine di Kbyte
DRAM	50 – 70 ns	alcuni Gbyte
Flash	0,005 – 0,05 ms	centinaia Gbyte
Dischi	5 – 20 ms	molti Tbyte

Nastri: decine di anni e Pbyte di dati



Livelli delle gerarchie di memoria e tecnologie

Livello	Tecnologia	Nome dell'Unità trasferita	Dimensione tipica dell'unità trasferita	Dimensione tipica della memoria
Cache (1° liv)	SRAM			16 KB – 64 KB
Cache (2° liv)	SRAM (+ lente e + grandi)	Blocco o linea	4 B – 512 B	128 KB– 8048 KB
Memoria Centrale	DRAM	Blocco	64 B – 4096 B	< ordine decine GB
Memoria Secondaria	Flash o Disco	Pagina	> 4 Kbyte	

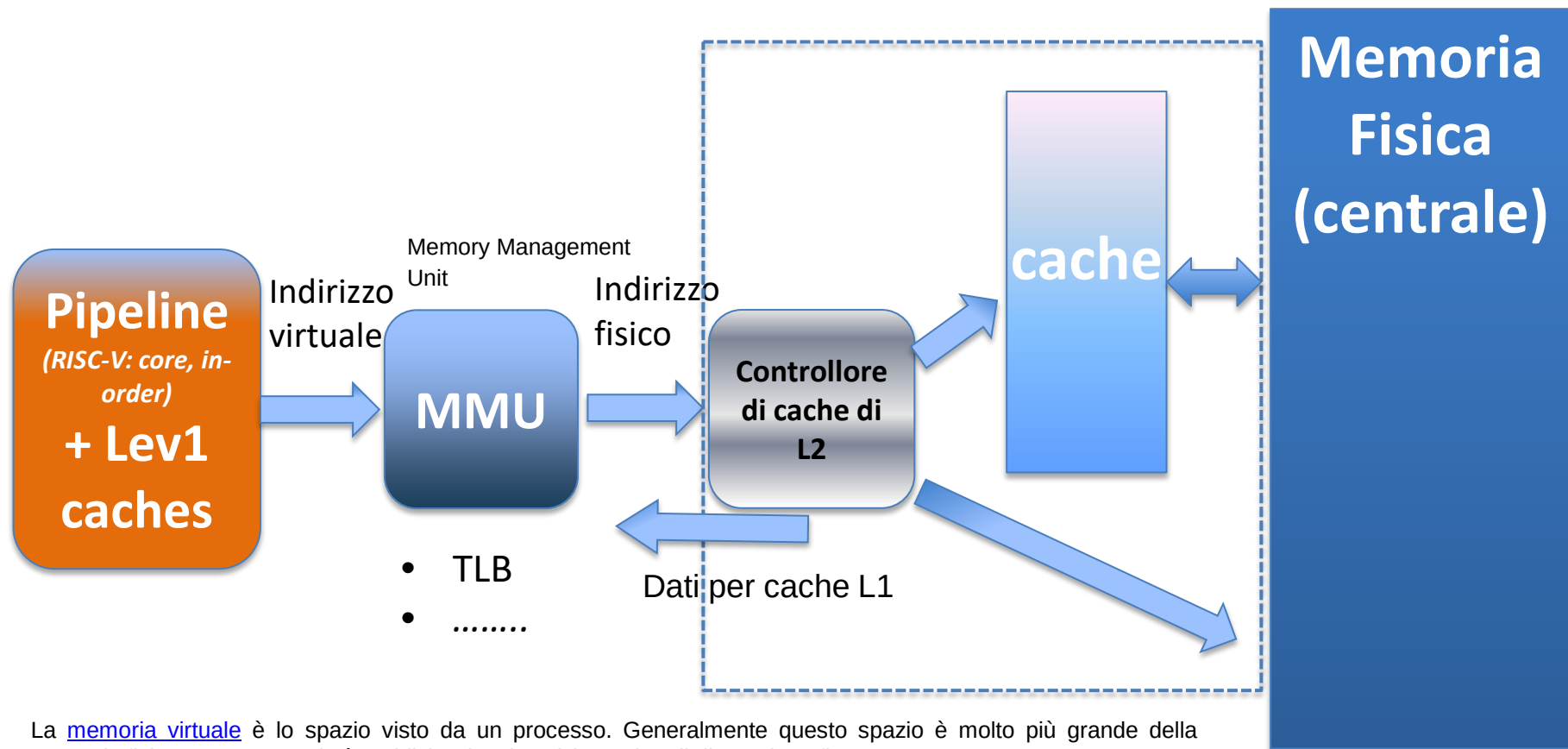


Gerarchia di Memoria ... il problema degli indirizzi

- Gli indirizzi di **memoria centrale** sono **indirizzi fisici**
- Gli indirizzi **generati dalla CPU** sono **indirizzi virtuali** e la *memoria virtuale* e la *memoria fisica* non coincidono
- Nella gerarchia di memoria è presente (almeno) un meccanismo di **traduzione automatica degli indirizzi**:
 - con la memoria virtuale, la CPU produce un **indirizzo virtuale** che viene tradotto da una combinazione di **elementi hardware e software** in un **indirizzo fisico**, che può essere utilizzato per accedere alla memoria centrale
 - ma non solo nella gerarchia di memoria è presente almeno un livello di cache e quindi è necessaria la traduzione automatica tramite **hardware** tra indirizzi della **memoria fisica** e indirizzi di **cache**



Processore, MMU e memoria fisica



La [memoria virtuale](#) è lo spazio visto da un processo. Generalmente questo spazio è molto più grande della memoria fisica. Questo spazio è suddiviso (paginato) in pagine di dimensione fissa.

La [page table](#) (tabella delle pagine), generalmente caricata in memoria centrale, tiene traccia di dove le pagine virtuali vengono caricate nella memoria fisica.

Per poter accedere a una locazione di memoria fisica occorre controllare che essa contenga la pagina virtuale voluta tramite il TLB e successivamente fare l'accesso tramite la gerarchia di cache. Il TLB ([Translation Lookaside Buffer](#) - memoria di traduzione a lato) o [Page Table cache](#) è una cache della [Page Table](#), cioè solamente un sottoinsieme del suo contenuto è memorizzato nel TLB.

Il TLB, come per la cache di memoria può essere "separato" Istruzioni (ITLB) e Dati (DTLB) o "unificato", come pure essere Direct Mapped o Set Associative o Fully Associative.

